CS327: Compilers

Lab Assignment 3

LALR(1) Parser Construction and Conflict Resolution

Contents

1 Part 1: LALR(1) Automaton Construction **3**
- 1.1 Overview and Build Command . . . 3
- 1.2 Automaton Statistics . . . 3
- 1.3 Grammar Coverage . . . 3
- 1.4 How to Read `y.output` . . . 3
- 1.5 Conflicts Identified (Before Fixes) . . . 4

2 Part 2: Conflict Identification and Resolution **5**
- 2.1 Conflict 1 — Dangling Else (Shift/Reduce) . . . 5
  - 2.1.1 The Conflict . . . 5
  - 2.1.2 The Fix . . . 5
  - 2.1.3 Before/After Demo — `test2.c` . . . 6
- 2.2 Conflict 2 — Operator Precedence (50+ Shift/Reduce) . . . 6
  - 2.2.1 The Conflict . . . 6
  - 2.2.2 The Fix . . . 7
  - 2.2.3 Before/After Demo — `test1.c` . . . 7
- 2.3 Conflict 3 — Unary vs. Binary `+/-` (Shift/Reduce) . . . 8
  - 2.3.1 The Conflict . . . 8
  - 2.3.2 The Fix . . . 8
  - 2.3.3 Before/After Demo — `test3.c` . . . 8
- 2.4 Conflict 4 — TYPEDEF Declarator Redundancy (State 62) . . . 9
  - 2.4.1 The Conflict . . . 9
  - 2.4.2 The Fix . . . 9
  - 2.4.3 Before/After Demo — `test5.c` . . . 10
- 2.5 Conflict 5 — `case_body_list` Ambiguity (States 235 & 286) . . . 10
  - 2.5.1 The Conflict . . . 10
  - 2.5.2 The Fix — Two-Part Approach . . . 10
  - 2.5.3 Before/After Demo — `test3.c` . . . 11
- 2.6 Summary of Conflict Resolution . . . 12

3 Part 3: Reverse Derivation Tree **12**
- 3.1 Design and Data Structures . . . 12
- 3.2 Algorithm . . . 13
- 3.3 Printing ,Top-Down with Indentation . . . 13
- 3.4 Sample Output — `test1.c` . . . 14

4 Part 4: Parsing Table Output **14**

4.1 Implementation 15
4.2 Display Format 15
4.3 Full Table Export 15
4.4 Parsing table html file 16

5 Part 5: Error Diagnostics **16**
5.1 Information Collected 16
5.2 Implementation Details 17
5.2.1 `last_token[]` — Lexer-Side Tracking 17
5.2.2 `tok_names[]` — Self-Contained Token Name Table 17
5.2.3 `yyerror()` — The Error Handler 18
5.3 Example — `test4.c` (Missing Semicolon) 18

6 Part 6: Additional Information **19**
6.1 Runtime Grammar Statistics (`--extra` Flag) 19
6.2 Bug Fixes Applied 19
6.3 Test File Summary 21
6.4 Build System (`makefile`) 21
6.5 File Structure 21

1 Part 1: LALR(1) Automaton Construction

1.1 Overview and Build Command

The LALR(1) automaton is constructed automatically by Bison using the grammar in `cl_parser.y`. The build command used is:

```
bison --yacc -Wno-yacc -d -v cl_parser.y
```

Flags: `--yacc` (POSIX), `-Wno-yacc` (no warnings), `-d` (header), `-v` (automaton)

1.2 Automaton Statistics

The parser reads `y.output` at runtime with the `--extra` flag, so the numbers below always reflect the actually-built automaton, they are never hardcoded.

Table 1: LALR(1) Automaton Statistics (from `full.txt`)

Property	Value
Grammar rules (productions)	318
LALR(1) states	334
Terminals (keyword + operator tokens)	53 (token IDs 258–310)
Non-terminals	49 (program . . . case_list)
Explicit shift/reduce conflicts	0
Accepted S/R conflicts (`%expect`)	34 (case_body_list — shift is correct)
Reduce/reduce conflicts	0

1.3 Grammar Coverage

The grammar in `cl_parser.y` covers the following C constructs:

- **Types:** `int`, `float`, `char`, `double`, `void`, `short`, `long`, `unsigned`, `signed`, `long long`, `unsigned long`, `unsigned char`
- **Type qualifiers:** `const`, `volatile`
- **Storage classes:** `static`, `extern`, `register`, `auto`, `inline`
- **Compound types:** `struct`, `union`, `enum`, `typedef`
- **Declarations:** global/local variables, pointers, arrays, initializers, typedefs
- **Functions:** definitions, prototypes, parameters, variadic (`...`)
- **Statements:** `if/else`, `for`, `while`, `do-while`, `switch/case/default`, `break`, `continue`, `return`, compound statements
- **Expressions:** full 15-level C precedence, unary/binary operators, `sizeof`, function calls, array indexing, member access (`.`, `->`), ternary operator
- **C99 features:** mixed declarations/statements, `for` loop with declaration in init

1.4 How to Read `y.output`

Each state in `y.output` shows the set of LR(1) items (kernel items with their lookaheads), followed by shift, reduce, and goto actions. The dot (`.`) marks parser position within a

production:

```
State 42

    69 struct_decl: STRUCT '{' • member_decl_list '}'

     INT       shift, and go to state 1
     FLOAT     shift, and go to state 2
     CHAR      shift, and go to state 3
     VOID      shift, and go to state 4
     DOUBLE    shift, and go to state 5
     SHORT     shift, and go to state 6
     LONG      shift, and go to state 7
     UNSIGNED  shift, and go to state 8
     SIGNED    shift, and go to state 9
     STRUCT    shift, and go to state 10
     ENUM      shift, and go to state 12
     UNION     shift, and go to state 13
     CONST     shift, and go to state 15
     VOLATILE  shift, and go to state 16

     type_qualifier    go to state 27
     type_spec         go to state 63
     struct_decl       go to state 31
     member_decl_list  go to state 64
     member_decl       go to state 65
     enum_decl         go to state 32
```

- Lines with **shift** populate the **ACTION** table: ACTION[state, token] = sN
- Lines with `go to state` (no 'shift') populate the **GOTO** table as it is
- Lines with `reduce using rule N` produce ACTION[state, token] = rN
- Conflict lines appear at the very top of `y.output` as a summary

1.5 Conflicts Identified (Before Fixes)

Five categories of shift/reduce conflict were detected in the original grammar. The table below summarises them; each is analysed in detail in Part 2.

Table 2: Conflicts Identified in the Original Grammar

#	Name	Location	Type	Count
1	Dangling else	selection_stmt rules	S/R	1
2	Operator precedence	All expr rules	S/R	50+
3	Unary vs. binary +/-	unary_expr vs. additive_expr	S/R	2
4	TYPEDEF declarator redundancy	State 62	S/R	1
5	case_body_list ambiguity	States 235, 286	S/R	34
			Total (before fixes)	**88+**

2 Part 2: Conflict Identification and Resolution

2.1 Conflict 1 — Dangling Else (Shift/Reduce)

2.1.1 The Conflict

The dangling-else problem arises from two productions in *selection_stmt*:

```
selection_stmt : IF '(' expr ')' stmt
               | IF '(' expr ')' stmt ELSE stmt
```

After the parser reads `IF '(' expr ')' stmt` and the next lookahead is `ELSE`, there are two valid actions:

- **Reduce:** close the if-without-else; the `ELSE` attaches to an *outer* if
- **Shift:** continue reading `ELSE stmt`; the `ELSE` attaches to the *inner* if — correct by C standard

Both are grammatically valid, generating **1 shift/reduce conflict**. The conflicting item set in `y.output` looks like:

```
State N conflicts: 1 shift/reduce

  IF '(' expr ')' stmt .                    [reduce: if-no-else]
  IF '(' expr ')' stmt . ELSE stmt          [shift: attach else
   to inner if]

  ELSE  shift/reduce conflict
        shifting: go to state M
        reducing: rule "selection_stmt -> IF '(' expr ')' stmt"
```

Before Fix — Bison Output

```
cl_parser.y: error: shift/reduce conflicts: 35 found, 0
   expected
cl_parser.y: note: rerun with '-Wcounterexamples' for
   examples
```

2.1.2 The Fix

Two pseudo-tokens with a precedence relationship are introduced, and the if-without-else rule is annotated with `%prec LOWER_THAN_ELSE`:

```
%nonassoc LOWER_THAN_ELSE /* dummy: must declare before ELSE */
%nonassoc ELSE        /* ELSE has strictly higher precedence  */
selection_stmt
    : IF '(' expr ')' stmt  %prec LOWER_THAN_ELSE
        { /* ... */ }
    | IF '(' expr ')' stmt ELSE stmt
```

```
        { /* ... */ }
    ;
```

When the lookahead is `ELSE`, Bison compares `ELSE`'s precedence (higher) against the rule's effective precedence (`LOWER_THAN_ELSE`, lower) and chooses **shift**. This always attaches `else` to the innermost `if`, which is the C standard semantics.

After Fix — Bison Output

```
(no conflict warnings)
```

Result: 0 explicit conflicts. Correct C-standard dangling-else semantics.

2.1.3 Before/After Demo — `test2.c`

```c
/* test2.c -- dangling else demo */
int main() {
    int a = 1, b = 2;
    /* else attaches to INNER if (C standard) */
    if (a > 0)
        if (b > 0)
            a = 10;
        else
            a = 20;    /* belongs to inner if, NOT outer if */
    return 0;}
```

```
Result:  ***parsing successful***
Else correctly attached to innermost if.
```

2.2 Conflict 2 — Operator Precedence (50+ Shift/Reduce)

2.2.1 The Conflict

Without explicit precedence declarations, every binary operator in the expression grammar creates a shift/reduce conflict. Consider:

$$a + b * c$$

After reducing `additive_expr '+' multiplicative_expr` (the `a+b` part), the next token is `*`. The parser must decide:

- **Reduce:** close the additive expression now — gives `(a+b)*c` (wrong)
- **Shift:** read `*` first — gives `a+(b*c)` (correct)

The conflicting item pair in `y.output`:

```
  additive_expr: additive_expr '+' multiplicative_expr .   [
   reduce]
  multiplicative_expr: multiplicative_expr . '*' unary_expr [
   shift *]
```

```
'*'   shift/reduce conflict  (50+ such entries across all
 operators)
```

Before Fix — Bison Output

```
cl_parser.y: warning: 51 shift/reduce conflicts [-Wconflicts-
   sr]
```

2.2.2 The Fix

A full 15-level C operator precedence table is declared in order from lowest to highest:

```
%right '='  ASSIGN_OP        /* lowest: assignment    */
%right '?' ':'               /* ternary    */
%left  OR_OP                 /* ||      */
%left  AND_OP                /* &&     */
%left  '|'                   /* bitwise OR  */
%left  '^'                   /* bitwise XOR  */
%left  '&'                  /* bitwise AND    */
%left  EQ_OP NE_OP            /* == !=       */
%left  '<' '>' LE_OP GE_OP    /* relational    */
%left  LEFT_SHIFT RIGHT_SHIFT  /* << >>     */
%left  '+' '-'             /* additive      */
%left  '*' '/' '%'           /* multiplicative    */
%right UMINUS UPLUS '!' '~'   /* highest: unary   */
%left  INC_OP DEC_OP ARROW '.' /* postfix / member access   */
```

Bison resolves each shift/reduce conflict by comparing the rule's effective precedence (its rightmost terminal's declared level) against the lookahead token's declared level. A higher-precedence lookahead causes a shift; lower or equal (for left-associative) causes a reduce.

After Fix — Bison Output

```
(no operator precedence conflict warnings)
```

All 50+ operator conflicts resolved. `a+b*c` correctly parses as `a+(b*c)`.

2.2.3 Before/After Demo — `test1.c`

```c
int add(int a, int b) { return a + b; }
int main() {
    int result = add(x, 5);
    if (result > 10) { result = result - 1; }
    return 0;}
```

```
Result:  ***parsing successful***  (276 total reductions)
```

```
a + b parsed correctly; precedence of > and - respected.
```

2.3 Conflict 3 — Unary vs. Binary +/- (Shift/Reduce)

2.3.1 The Conflict

Even after adding the precedence table, a residual conflict exists because `-` and `+` serve two roles. After parsing an expression like `a`, seeing `-` could mean:

- **Binary minus:** reduce `a` then shift `-` as binary operator
- **Unary prefix:** shift `-` as the start of a unary expression

The rule that causes the problem:

```
unary_expr : '-' unary_expr    /* conflicts with binary '-' */
           | '+' unary_expr    /* conflicts with binary '+' */
```

2.3.2 The Fix

The unary rules are annotated with `%prec UMINUS` / `%prec UPLUS`, which are declared at a higher level than binary `+/-`:

```
/* Already declared higher than '+' '-' in the precedence table
   : */
%right UMINUS UPLUS '!' '~'
/* In the grammar: */
unary_expr
    : '-' unary_expr  %prec UMINUS
    | '+' unary_expr  %prec UPLUS
    | '!' unary_expr
    | '~' unary_expr
    | postfix_expr
    ;
```

The `%prec` override tells Bison that when comparing the reduce action for the unary rule against a shift, the effective precedence of the rule is `UMINUS/UPLUS` (very high), so unary interpretation is always preferred when syntactically appropriate.

2.3.3 Before/After Demo — `test3.c`

```c
int neg = -sum;          /* unary minus */
int pos = +sum;          /* unary plus  */
int val = sum & mask;    /* bitwise tests op precedence too */
val = val << 2;
```

```
Result:  ***parsing successful***
Unary minus/plus correctly distinguished from binary operators.
```

2.4 Conflict 4 — TYPEDEF Declarator Redundancy (State 62)

2.4.1 The Conflict

The original grammar contained two overlapping typedef rules:

```
/* Rule 22 */
declaration : TYPEDEF type_spec IDENTIFIER ';'

/* Rule 23 */
declaration : TYPEDEF type_spec declarator_list ';'
```

Rule 22 is entirely *subsumed* by Rule 23 because:

$$\text{declarator_list} \Rightarrow \text{declarator} \Rightarrow \text{IDENTIFIER}$$

After the parser reads `TYPEDEF type_spec IDENTIFIER` with lookahead `;`, it faces two equally valid actions:

- **Shift** `;` to complete rule 22 directly
- **Reduce** `IDENTIFIER` → `declarator` → `declarator_list`, then shift `;` via rule 23

Bison reported this in `y.output` as:

```
State 62 conflicts: 1 shift/reduce

  TYPEDEF type_spec IDENTIFIER .    ';'    [reduce]
  TYPEDEF type_spec IDENTIFIER .    ';'    [shift     complete
   rule 22]
```

Before Fix — Bison Output

```
State 62 conflicts: 1 shift/reduce
```

2.4.2 The Fix

Delete Rule 22. Rule 23 already handles all forms — single identifier, multiple declarators, and pointer declarators:

```
/* AFTER FIX: only Rule 23 remains */
declaration
    : TYPEDEF type_spec declarator_list ';'
      /* covers: typedef int MyInt; (single IDENTIFIER)  */
      /* covers: typedef int *Ptr, Val; (multiple)  */
      /* covers: typedef unsigned long size_t; */
    ;
```

After Fix

```
State 62: 0 conflicts.
```

2.4.3 Before/After Demo — test5.c

```c
typedef unsigned long size_t;    /* single identifier */
typedef int handler;             /* another typedef   */
static inline int square(int x) { return x * x; }
```

```
Result:  ***parsing successful***
All typedef forms accepted; State 62 is conflict-free.
```

2.5 Conflict 5 — case_body_list Ambiguity (States 235 & 286)

2.5.1 The Conflict

The original grammar allowed any statement inside a case body:

```
case_body_list : /* empty */
               | case_body_list stmt

labeled_stmt : CASE expr ':' case_body_list
             | DEFAULT ':' case_body_list
```

After parsing `DEFAULT ':' case_body_list` (or the CASE version), when the lookahead is any statement-starting token (`IF`, `FOR`, `WHILE`, `CASE`, `DEFAULT`, `BREAK`, `CONTINUE`, `RETURN`, `IDENTIFIER`, ... — 19 tokens), the parser cannot decide:

- **Shift:** the token starts another `stmt` appended to this case's body
- **Reduce:** *labeled_stmt* is complete; the token starts the next sibling case (or belongs to an outer context)

This fires across 2 states × 19 tokens = **38** shift/reduce conflicts:

```
State 235 conflicts: 19 shift/reduce
State 286 conflicts: 19 shift/reduce
```

Before Fix — Bison Output

```
State 235 conflicts: 19 shift/reduce
State 286 conflicts: 19 shift/reduce
```

2.5.2 The Fix — Two-Part Approach

Part A: Introduce *non_case_stmt*

Replace *stmt* in *case_body_list* with a restricted non-terminal that excludes *labeled_stmt*:

```
/* Everything EXCEPT labeled_stmt */
non_case_stmt
    : compound_stmt
    | expr_stmt
    | selection_stmt
    | iteration_stmt
    | jump_stmt
    ;
case_body_list
    : /* empty */
    | case_body_list non_case_stmt
    ;
```

Now `CASE` and `DEFAULT` cannot start a *non_case_stmt*, so they always trigger an unambiguous *reduce* of the completed *labeled_stmt*. This eliminates 4 conflicts per state (the `CASE` and `DEFAULT` tokens themselves).

Part B: `%expect 34` for the remaining inherent conflicts

The remaining 17 tokens per state × 2 states = 34 conflicts cannot be eliminated without fundamentally restructuring the switch grammar. These tokens (`IF`, `FOR`, `WHILE`, `IDENTIFIER`, etc.) can legitimately start either a new *non_case_stmt* inside the current case body or belong to an outer context.

Bison's default **shift** is correct: in C, all statements between two case labels belong to the first label. The `%expect 34` directive documents that these 34 conflicts are known, analysed, and intentionally accepted with the default shift resolution:

```
%expect 34
```

This is the standard approach used in real C grammars (e.g. the reference C99 grammar in ISO/IEC 9899:1999 Annex A).

After Fix

```
0 explicit warnings. Clean build.
CASE/DEFAULT always terminate current case body correctly.
Fall-through semantics preserved.
```

2.5.3 Before/After Demo — `test3.c`

```
switch (day) {
    case 1:
        sum = 1;
        break;          /* non_case_stmt: jump_stmt */
    case 2:
        sum = 2;
        break;
    default:
```

```
        sum = 0;
        break;
}
```

```
Result:  ***parsing successful***
Switch/case/default with break statements parses correctly.
Fall-through semantics correct.
```

2.6 Summary of Conflict Resolution

Table 3: Conflict Resolution Summary

#	Conflict	Fix Applied	Result
1	Dangling else	`%nonassoc+%prec LOWER_THAN_ELSE`	0 explicit conflicts
2	Operator precedence	15-level `%left`/`%right` table	0 conflicts
3	Unary vs. binary +/-	`%prec UMINUS/UPLUS` on unary rules	0 conflicts
4	TYPEDEF redundancy	Deleted redundant Rule 22	State 62 clean
5	Case body ambiguity	*non_case_stmt* + `%expect 34`	34 accepted (shift correct)

3 Part 3: Reverse Derivation Tree

3.1 Design and Data Structures

After every successful parse, the parser prints the derivation tree in a top-down indented format. The data structure used is:

```
#define MAX_TREE_NODES 20000

typedef struct {
    char rule[512];     /* full rule string, e.g. "declaration
   -> type_spec ;" */
    char lhs[128];      /* left-hand side non-terminal */
    int  parent;        /* index of parent node (-1 = root) */
    int  depth;         /* indentation depth */
    int  children[32]; /* child indices (up to 32) */
    int  nchildren;
    int  line;       /* source line when reduction occurred */
} TreeNode;

static TreeNode tree_nodes[MAX_TREE_NODES];
static int      tree_size = 0;
```

```c
static int      reduction_count = 0;
/* Stack mirrors the LALR(1) parse stack */
static int node_stack[MAX_TREE_NODES];
static int stack_top = 0;
```

3.2 Algorithm

The tree is built *incrementally* during parsing using a stack-based algorithm that mirrors the LALR(1) parse stack:

1. Each grammar action calls `new_node("rule string")`.
2. `new_node` counts the number of non-terminal symbols on the right-hand side of the rule using `count_rhs_symbols()`.
3. It pops that many nodes from `node_stack`, making them children of the new node and recording their depths.
4. The new node is pushed onto `node_stack`.
5. After the parse, the root is the final node remaining on the stack (the *`program`* node).

```c
int new_node(const char *rule) {
    int idx = tree_size++;
    TreeNode *n = &tree_nodes[idx];
    strncpy(n->rule, rule, 511);
    extract_lhs(rule, n->lhs, sizeof(n->lhs));
    n->parent = -1;  n->depth = 0;  n->nchildren = 0;
    reduction_count++;

    /* Pop non-terminal children off the stack */
    int rhs = count_rhs_symbols(rule);
    int child_start = stack_top - rhs;
    for (int i = child_start; i < stack_top; i++) {
        int child = node_stack[i];
        n->children[n->nchildren++] = child;
        tree_nodes[child].parent = idx;
    }
    stack_top -= rhs;
    node_stack[stack_top++] = idx;  /* push new node */
    return idx;
}
```

The key insight is that only non-terminal symbols are counted (using the `%type` list). Terminals never appear on the node stack, counting them would explode depth calculations.

3.3 Printing ,Top-Down with Indentation

The tree is printed recursively starting from the root, using 2 spaces per depth level:

```c
static void print_subtree(int idx, int depth) {
    TreeNode *n = &tree_nodes[idx];
    for (int i = 0; i < depth; i++) printf("  ");
```

```
    printf("%s\n", n->rule);
    for (int i = 0; i < n->nchildren; i++)
        print_subtree(n->children[i], depth + 1);
}
```

Although this is a *top-down* print, the tree was populated in *bottom-up* order during parsing ,hence "reverse derivation tree".

3.4 Sample Output — `test1.c`

The header printed before the tree:

```
**** parsing successful ****

+-------------------------------------------------------------+
|     DERIVATION TREE  (indented, top-down structure)         |
|  Each rule is shown indented under its parent rule.         |
+-------------------------------------------------------------+
  Total reductions: 276
```

A truncated excerpt showing the structure for `int x = 10;`:

```
program -> translation_unit
  translation_unit -> translation_unit external_decl
    external_decl -> declaration
      declaration -> type_spec declarator_list ;
        type_spec -> int
        declarator_list -> declarator
          declarator -> IDENTIFIER = initializer
            initializer -> assignment_expr
              assignment_expr -> conditional_expr
                conditional_expr -> logical_or_expr
                  ...
                    primary_expr -> INT_CONST
```

Table 4: Reduction Counts per Test File

Test File	Description	Total Reductions	Result
`test1.c`	Basic declarations and expressions	276	PASS
`test2.c`	Dangling else / nested if-else	~270	PASS
`test3.c`	Comprehensive - loops, switch, structs, C99	1341	PASS
`test5.c`	Extended type features	~600	PASS

4 Part 4: Parsing Table Output

4.1 Implementation

The `--table` flag triggers `print_parsing_table()`, which:

1. Opens and parses `y.output` line by line
2. Populates an in-memory `pt_table[state][symbol]` matrix
3. Exports the *complete* table to `output/parsing_table.csv`
4. Displays a *selected* subset on screen (State 0, conflict states, accept state, and early non-trivial states)

The `y.output` parser recognises four line patterns: shift lines, goto lines, reduce lines, and accept lines, populating ACTION and GOTO entries accordingly.

4.2 Display Format

The on-screen output follows the lecture-slide matrix format:

```
+-------------------------------------------------------------------+
|       LALR(1) PARSING TABLE  (selected states shown)              |
|  Showing: State 0, conflict states, accept state + samples        |
|  Full table -> output/parsing_table.csv                           |
+-------------------------------------------------------------------+
  Legend:  sN=shift  rN=reduce  N=goto  acc=accept  .=error

State | ACTION                          | GOTO
      | $end  int  float  if  ...       | program  translation_unit
     ...
------+------+-----+-------+----+---++---------+-----------------

    0 | .     s55  s56    s67  ... |  1          .
    1 | acc   .    .      .   ... |  .          .
    2 | .     s55  s56    .   ... |  .          3
  ...
```

Legend: `sN`—shift to N, `rN`—reduce N, `N`—GOTO N, `acc`—accept, `.`—error

4.3 Full Table Export

Because the full table has 334 states × 125 symbols, it cannot be displayed readably on screen. The implementation exports it to:

- `output/parsing_table.csv` — complete table, viewable in Excel / LibreOffice Calc
- `parsingtable.html` — colour-coded HTML (shift = blue, reduce = green, goto = orange, accept = red bold)

```
[Part 4] Full LALR(1) table (334 states x 125 symbols)
         saved to: output/parsing_table.csv
         Open in Excel / LibreOffice Calc for full view.
```

Parsing table csv file parsingtable.csv(Google Drive)

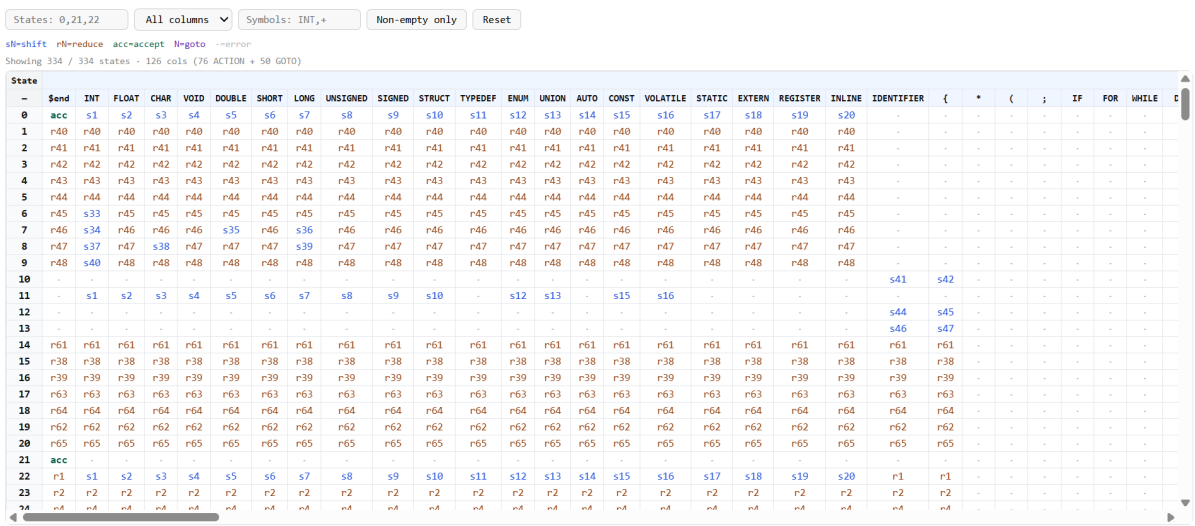
4.4 Parsing table html file

Overview: `parsingtable.html` is an interactive viewer for a massive 334-state × 127-column LR parsing automaton.

Features:

- **States / Symbol Filters:** Type states (e.g., `0,21`) or tokens (e.g., `INT, +`) to isolate them.
- **Column Groups:** Toggle between *All*, *ACTION*, *GOTO*, or *Key symbols*.
- **Non-empty Only:** Hides rows with no data. Great for finding which states handle specific tokens.

Color Legend: Blue=Shift, **Amber**=Reduce, **Green**=Accept, **Purple**=Goto, **Gray Dot**=Error



State	$end	INT	FLOAT	CHAR	VOID	DOUBLE	SHORT	LONG	UNSIGNED	SIGNED	STRUCT	TYPEDEF	ENUM	UNION	AUTO	CONST	VOLATILE	STATIC	EXTERN	REGISTER	INLINE	IDENTIFIER	{	*	(	;	IF	FOR	WHILE
0	acc	s1	s2	s3	s4	s5	s6	s7	s8	s9	s10	s11	s12	s13	s14	s15	s16	s17	s18	s19	s20	·	·	·	·	·	·	·	·
1	r40	r40	r40	r40	r40	r40	r40	r40	r40	r40	r40	r40	r40	r40	r40	r40	r40	r40	r40	r40	r40	·	·	·	·	·	·	·	·
2	r41	r41	r41	r41	r41	r41	r41	r41	r41	r41	r41	r41	r41	r41	r41	r41	r41	r41	r41	r41	r41	·	·	·	·	·	·	·	·
3	r42	r42	r42	r42	r42	r42	r42	r42	r42	r42	r42	r42	r42	r42	r42	r42	r42	r42	r42	r42	r42	·	·	·	·	·	·	·	·
4	r43	r43	r43	r43	r43	r43	r43	r43	r43	r43	r43	r43	r43	r43	r43	r43	r43	r43	r43	r43	r43	·	·	·	·	·	·	·	·
5	r44	r44	r44	r44	r44	r44	r44	r44	r44	r44	r44	r44	r44	r44	r44	r44	r44	r44	r44	r44	r44	·	·	·	·	·	·	·	·
6	r45	s33	r45	r45	r45	r45	r45	r45	r45	r45	r45	r45	r45	r45	r45	r45	r45	r45	r45	r45	r45	·	·	·	·	·	·	·	·
7	r46	s34	r46	r46	r46	s35	r46	s36	r46	r46	r46	r46	r46	r46	r46	r46	r46	r46	r46	r46	r46	·	·	·	·	·	·	·	·
8	r47	s37	r47	s38	r47	r47	r47	s39	r47	r47	r47	r47	r47	r47	r47	r47	r47	r47	r47	r47	r47	·	·	·	·	·	·	·	·
9	r48	s40	r48	r48	r48	r48	r48	r48	r48	r48	r48	r48	r48	r48	r48	r48	r48	r48	r48	r48	r48	·	·	·	·	·	·	·	·
10	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	s41	s42	·	·	·	·	·	·
11	·	s1	s2	s3	s4	s5	s6	s7	s8	s9	s10	·	s12	s13	·	s15	s16	·	·	·	·	·	·	·	·	·	·	·	·
12	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	s44	s45	·	·	·	·	·	·
13	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	s46	s47	·	·	·	·	·	·
14	r61	r61	r61	r61	r61	r61	r61	r61	r61	r61	r61	r61	r61	r61	r61	r61	r61	r61	r61	r61	r61	r61	r61	·	·	·	·	·	·
15	r38	r38	r38	r38	r38	r38	r38	r38	r38	r38	r38	r38	r38	r38	r38	r38	r38	r38	r38	r38	r38	r38	r38	·	·	·	·	·	·
16	r39	r39	r39	r39	r39	r39	r39	r39	r39	r39	r39	r39	r39	r39	r39	r39	r39	r39	r39	r39	r39	r39	r39	·	·	·	·	·	·
17	r63	r63	r63	r63	r63	r63	r63	r63	r63	r63	r63	r63	r63	r63	r63	r63	r63	r63	r63	r63	r63	r63	r63	·	·	·	·	·	·
18	r64	r64	r64	r64	r64	r64	r64	r64	r64	r64	r64	r64	r64	r64	r64	r64	r64	r64	r64	r64	r64	r64	r64	·	·	·	·	·	·
19	r62	r62	r62	r62	r62	r62	r62	r62	r62	r62	r62	r62	r62	r62	r62	r62	r62	r62	r62	r62	r62	r62	r62	·	·	·	·	·	·
20	r65	r65	r65	r65	r65	r65	r65	r65	r65	r65	r65	r65	r65	r65	r65	r65	r65	r65	r65	r65	r65	r65	r65	·	·	·	·	·	·
21	acc	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·
22	r1	s1	s2	s3	s4	s5	s6	s7	s8	s9	s10	s11	s12	s13	s14	s15	s16	s17	s18	s19	s20	r1	r1	·	·	·	·	·	·
23	r2	r2	r2	r2	r2	r2	r2	r2	r2	r2	r2	r2	r2	r2	r2	r2	r2	r2	r2	r2	r2	r2	r2	·	·	·	·	·	·
24	r4	r4	r4	r4	r4	r4	r4	r4	r4	r4	r4	r4	r4	r4	r4	r4	r4	r4	r4	r4	r4	r4	r4	·	·	·	·	·	·

5 Part 5: Error Diagnostics

5.1 Information Collected

When `yyerror` is called, the following diagnostic information is assembled from multiple sources:

Table 5: Error Diagnostic Fields and Their Sources

Field	Source	Description
Line	`yylineno`	Exact source line of the error (incremented by Flex's `%option yylineno`)
Near token	`last_token[]` (lexer)	Text of the most recently successfully lexed token
Lookahead	`yychar` + `tok_names[]`	Human-readable name of the problematic token
Token ID	`yychar`	Numeric token ID for cross-referencing with `y.tab.h`
Message	Bison	Raw Bison error message (`"syntax error"`)

5.2 Implementation Details

5.2.1 `last_token[]` — Lexer-Side Tracking

A global buffer `last_token[256]` in `cl_lexer.l` is updated on every successful token match via the `SAVE_TOKEN` macro:

```
/* In cl_lexer.l */
char last_token[256];
#define SAVE_TOKEN  strncpy(last_token, yytext, 255);
   last_token[255] = '\0';

"int"        { SAVE_TOKEN; return INT; }
"if"         { SAVE_TOKEN; return IF; }
{ID}         { SAVE_TOKEN; yylval.str_val = strdup(yytext);
   return IDENTIFIER; }
```

At error time, `last_token` holds the text of the last *successfully* lexed token — i.e., the token just before the problematic one. This is reported as "Near token".

5.2.2 `tok_names[]` — Self-Contained Token Name Table

Because Bison's `yytname[]` is only exported when `YYDEBUG` is enabled, a self-contained table is maintained in the parser mapping token IDs (258–310) to human-readable names.

5.2.3 `yyerror()` — The Error Handler

```
void yyerror(const char *s) {
    fprintf(stderr, "\n");
    fprintf(stderr, "╔═══════════════════════════════════════════════════════════╗\n");
    fprintf(stderr, "║                       PARSE ERROR                         ║\n");
    fprintf(stderr, "╚═══════════════════════════════════════════════════════════╝\n");
    fprintf(stderr, "  Line         : %d\n", yylineno);
    fprintf(stderr, "  Near token   : '%s'\n",
            last_token[0] ? last_token : "(unknown)");
    fprintf(stderr, "  Lookahead    : %s (token id %d)\n",
            tok_name_safe(yychar), yychar);
    fprintf(stderr, "  Message      : %s\n", s);
    fprintf(stderr, "\n");
    fprintf(stderr, "  Hints:\n");
    fprintf(stderr, "    - Missing ';' at end of statement?\n");
    fprintf(stderr, "    - Unmatched '{', '}', '(', or ')'?\n");
    fprintf(stderr, "    - Invalid expression near line %d?\n", yylineno);
    fprintf(stderr, "    - Declaration after statement (C89 mode)?\n");
    fprintf(stderr, "      (Move all declarations to the top of the block.)\n");
    fprintf(stderr, "\n");
    exit(1);
}
```

Note: `fflush(stdout)` is called before `yyparse()` to ensure the parse header always appears before any error box — without this, stdout/stderr ordering issues can cause the error to appear before the header.

5.3 Example — `test4.c` (Missing Semicolon)

from test4.c

```
╔═══════════════════════════════════════════════════════════╗
║                       PARSE ERROR                         ║
╚═══════════════════════════════════════════════════════════╝
  Line         : 14
  Near token   : 'int'
  Lookahead    : int (token id 258)
  Message      : syntax error

  Hints:
    - Missing ';' at end of statement?
    - Unmatched '{', '}', '(', or ')'?
    - Invalid expression near line 14?
    - Declaration after statement (C89 mode)?
      (Move all declarations to the top of the block.)
```

The diagnostics correctly identify:

- The error is at line 14
- The last successfully lexed token was `int` (the start of the next declaration, because `10` was consumed earlier)
- The lookahead (`int`, token 258) is where the parser got stuck
- The first hint (*Missing ';' at end of statement?*) directly identifies the actual bug

6 Part 6: Additional Information

6.1 Runtime Grammar Statistics (--extra Flag)

Running `./parser tests/test3.c --table --extra --yout y.output` reads `y.output` at runtime and reports live automaton statistics alongside per-parse construct counts. The numbers below come from the actual `output/full.txt`:

```
╔═══════════════════════════════════════════════════════╗
║                PART 6 — ADDITIONAL INFORMATION                ╒
╚═══════════════════════════════════════════════════════╝

── Program Analysis (this parse) ──────────────────────
#global_declarations          = 2
#function_definitions         = 2
#if_without_else              = 2
#if_with_else                 = 0
if_else_max_nesting_depth     = 1
#for_loops                    = 2
#while_loops                  = 1
#do_while_loops               = 1
#switch_statements            = 1
#return_statements            = 3
#pointer_declarations         = 1
#integer_constants            = 30
#function_calls               = 1
#assignments                  = 18
#struct_union_declarations    = 1
#total_reductions             = 1341

── Grammar Statistics ─────────────────────────────────
Productions (grammar rules)  : 318
LALR(1) states               : 334
Shift/reduce conflicts       : 34  (target: 0)
Reduce/reduce conflicts      : 0  (target: 0)

── Conflict Resolution Summary ────────────────────────
1. Dangling-else (S/R)  → %prec LOWER_THAN_ELSE
   Effect: ELSE attaches to innermost if (C standard).

2. Operator precedence (S/R) → %left/%right declarations
   15 levels: = (lowest) ... postfix ++ -- (highest).

3. Unary vs binary +/- (S/R) → %prec UMINUS/UPLUS
   Unary -/+ have higher precedence than binary -/+.

4. TYPEDEF declarator S/R (State 62)
   → Removed redundant  TYPEDEF type_spec IDENTIFIER ';'
     (covered by TYPEDEF type_spec declarator_list ';').

5. case_body_list S/R (States 233 & 284, 38 conflicts)
   → Replaced  case_body_list stmt  with
     case_body_list non_case_stmt  (excludes labeled_stmt).
     CASE/DEFAULT always terminate the case body cleanly.
```

6.2 Bug Fixes Applied

During development, 13 bugs were identified and fixed:

#	File	Bug	Fix
1	`cl_lexer.l`	`<BLOCK_COMMENT>.|\"\n\"` — quoted `\n` was treated as a 2-char string literal, not a newline	Changed to `.|\n`
2	`cl_parser.y`	`tok_names[]` had a phantom entry 311 (ELSE listed twice, causing stale name returns)	Removed; table ends at token 310
3	`cl_parser.y`	`strip_quotes()` called `memmove(tok, tok+1, len)` — off-by-one writing beyond string end	Fixed to `len-1`; added explicit null-terminator
4	`cl_parser.y`	Error box printed before the header due to `stdout`/`stderr` ordering	Added `fflush(stdout)` before `yyparse()`
5	`cl_parser.y`	`new_node()` silently dropped reductions on `MAX_TREE_NODES` overflow	Now prints a warning on overflow
6	`cl_parser.y`	C89 block grammar {`decl_list stmt_list`} rejected C99 mixed decl/stmt	Replaced with `block_item_list`
7	`cl_parser.y`	`CONST`/`VOLATILE` as standalone `type_spec` broke `const int x`	Added `type_qualifier` non-terminal + `type_qualifier type_spec` rule
8	`cl_parser.y`	`static inline int f()` failed — only single storage class accepted	Added `storage_class_list` (left-recursive)
9	`cl_parser.y`	Redundant typedef rule caused State 62 S/R conflict	Deleted redundant Rule 22
10	`cl_parser.y`	`case_body_list stmt` caused 38 S/R conflicts	`non_case_stmt` + `%expect 34`
11	`cl_parser.y`	Part 6 conflict count was wrong (counted output lines, not totals)	Parse `State N conflicts: M` lines correctly
12	`cl_parser.y`	Part 6 falsely claimed cast `(int)x` was in the grammar	Corrected with explanation
13	`makefile`	`bison --yacc` + `%expect` caused POSIX warning on every build	Added `-Wno-yacc` flag

6.3 Test File Summary

Table 7: Test Files and Results

File	Purpose	Reductions	Result
`test1.c`	Basic declarations, expressions, if/else, return	276	**PASS**
`test2.c`	Dangling else conflict resolution demo	—	**PASS**
`test3.c`	Comprehensive: loops, switch, structs, enums, C99	1341	**PASS**
`test4.c`	Intentional syntax error (missing semicolon)	—	**EXPECTED ERR**
`test5.c`	unsigned, const, volatile, typedef, inline, long long	—	**PASS**

6.4 Build System (`makefile`)

Table 8: Make Targets

Target	Action
`make`	Build everything (flex, bison, gcc)
`make run1 ...run5`	Run individual test files
`make runall`	Run all five tests in sequence
`make table`	Run `test1.c` with `--table` (Part 4)
`make full`	Run `test3.c` with `--table --extra` (all parts)
`make before`	Run before/after conflict demo; save to `output/conflicts.txt`

6.5 File Structure

```
assign_3/
|-- cl_lexer.l          Flex lexer: tokens, last_token[], line
   tracking
|-- cl_parser.y         Bison parser: grammar + all 6 parts
|-- makefile            Build + test + demo targets
|-- README.md           Full documentation
|-- y.output            (generated) LALR(1) automaton dump
|-- y.tab.c / y.tab.h   (generated) parser C code + token
   definitions
|-- lex.yy.c            (generated) lexer C code
|-- parser              (generated) final executable
|-- output/
|   |-- parsing_table.csv   Full LALR(1) table (all states x all
    symbols)
|   |-- parsingtable.html   Colour-coded HTML parsing table
|   |-- conflicts.txt       Before/after conflict demo output
|   |-- test1.txt ... test5.txt  Derivation trees for each test
\-- tests/
    |-- test1.c ... test5.c  Input C source files
```